Signal Processing Coding Assignment

Name- Adorn Benny George

Reg. No- RA2311053010149

Course- ECE-DS

Section- L

Faculty name- Dr. Uday Kumar

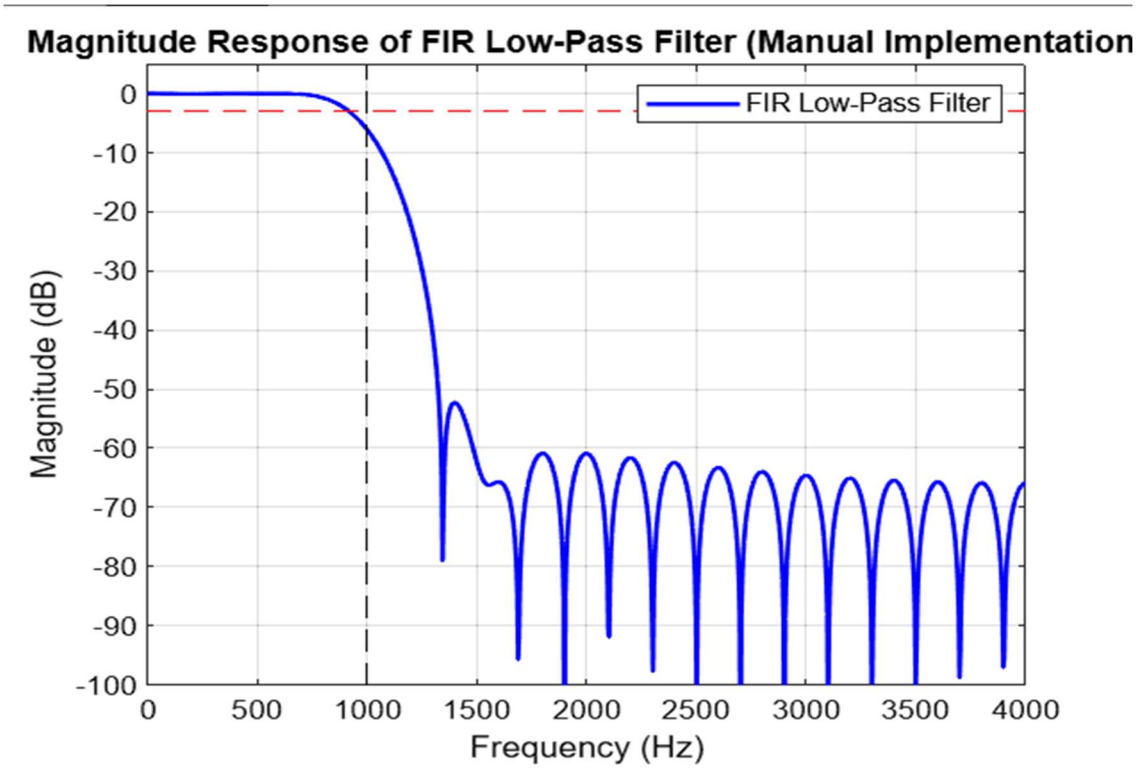
Signal Processing Assignment-3

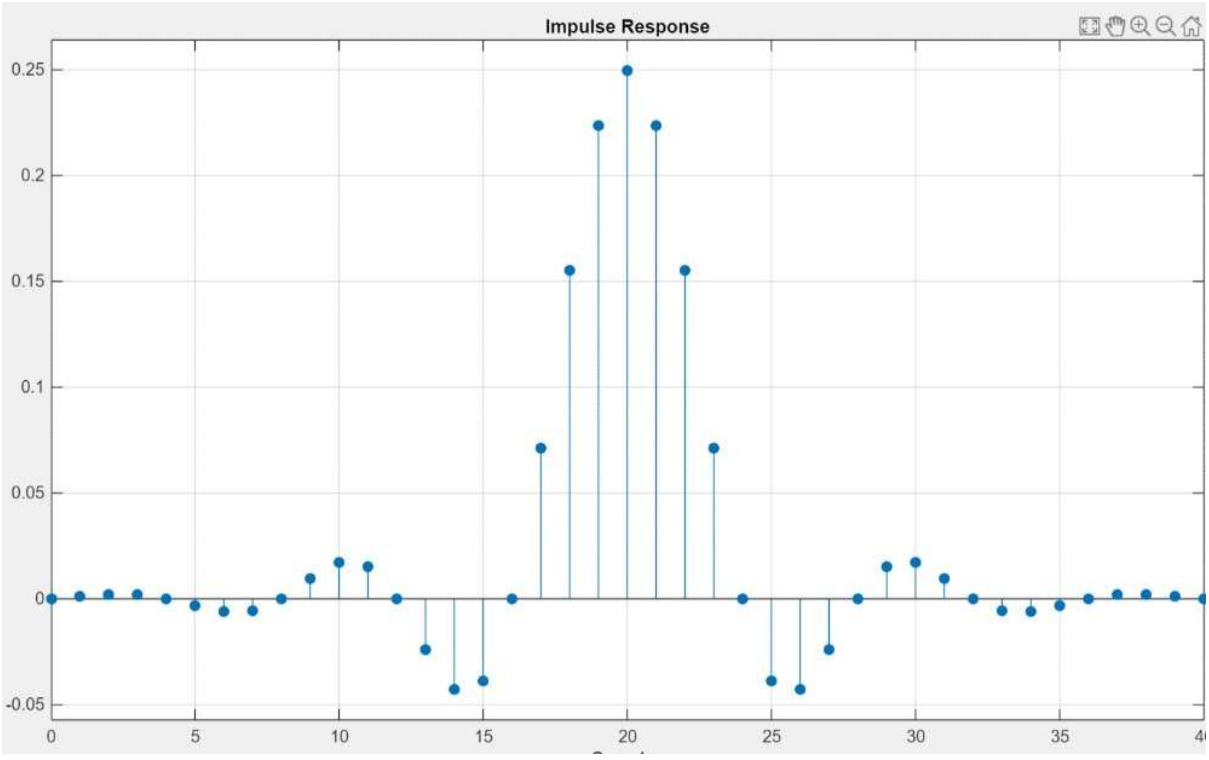
1.

```
fs = 8000;          % Sampling frequency in Hz
fc = 1000;          % Cutoff frequency in Hz
order = 40;         % Filter order
nyquist = fs / 2;    % Nyquist frequency

wc = fc / nyquist;   % Normalized cutoff (0 to 1)

N = order + 1;
n = 0:N-1;
w = 0.54 - 0.46 * cos(2 * pi * n / (N - 1));   % Hamming window

% Ideal impulse response of low-pass filter (sinc function)
m = n - (N - 1) / 2;
hd = wc * sinc(wc * (m));

b = hd .* w; % Apply window to ideal impulse response

f = linspace(0, fs/2, 1024); % frequency response
H = zeros(size(f));
```

```
for k = 1:length(f)
    omega = 2 * pi * f(k) / fs;
    H(k) = sum(b .* exp(-1j * omega * (0:N-1)));
end

% Plot the magnitude response in dB
figure;
plot(f, 20*log10(abs(H)), 'b', 'LineWidth', 1.5);
grid on;
title('Magnitude Response of FIR Low-Pass Filter (Manual Implementation)');
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
xlim([0 fs/2]);
ylim([-100 5]);

hold on;
plot([fc fc], [-100 5], '--k');       % cutoff line
plot([0 fs/2], [-3 -3], '--r');       % -3 dB line
legend('FIR Low-Pass Filter');
```

Output-


Magnitude Response of FIR Low-Pass Filter (Manual Implementation
FIR Low-Pass Filter
Magnitude (dB)
0
-10
-20
-30
-40
-50
-60
-70
-80
-90
-100
0
500
1000
1500
2000
2500
3000
3500
4000
Frequency (Hz)


Impulse Response
0.25
0.2
0.15
0.1
0.05
0
-0.05
0
5
10
15
20
25
30
35

2.

1.

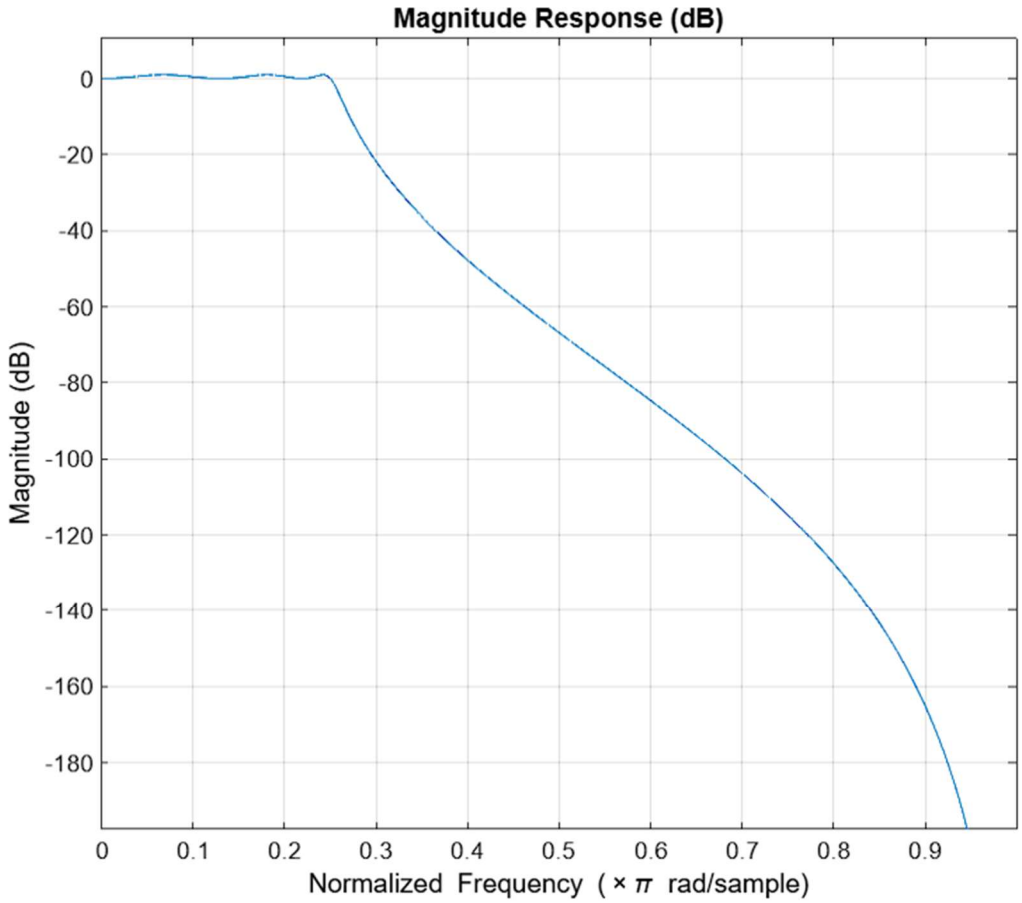
Low pass filter-

```
fs = 8000;          % Sampling frequency
fp = 1000;          % Passband frequency (Hz)
fsb = 1500;         % Stopband frequency (Hz)
Ap = 1;             % Passband attenuation (dB)
As = 40;            % Stopband attenuation (dB)

Wp = 2 * fs * tan(pi * fp / fs); %  Prewarped passband frequency
Ws = 2 * fs * tan(pi * fsb / fs); % Prewarped stopband frequency
ws = Ws / Wp; %  Normalized stopband frequency

% Computing Filter Order (n)
ep = sqrt(10^(Ap/10) - 1); %  Ripple factor
A = 10^(As/20);                 % Attenuation factor
n = acosh(sqrt((A^2 - 1)/ep^2)) / acosh(ws);  % Order
n = ceil(n);  % Rounding up to nearest integer

% Poles of Analog Chebyshev Type I Prototype
```

```
beta = asinh(1/ep)/n;
p = zeros(1, n);    % initialize pole array
for k = 1:n
   theta = pi * (2*k - 1) / (2*n);   % pole angle
   sigma = -sinh(beta) * sin(theta);  %  real part of pole
   omega = cosh(beta) * cos(theta);  %  imaginary part of pole
   p(k) = Wp * (sigma + 1j * omega);   %  scale poles by Wp
end

% Analog Low-Pass Transfer Function
a = real(poly(p)); %denominator
b = [a(end)];  %numerator

N = max(length(a), length(b)) - 1;  % bilinear transformation
   [a,b] = eqtflength(a,b);  % Equalize lengths

   D = zeros(N+1,N+1);
   for n = 0:N
      for k = 0:n
         D(n+1,k+1) = nchoosek(n,k)*(1)^(k)*(-1)^(n-k);
      end
   end
   T = 2/fs;
```

```
    az = real(D * (a(:) .* T.^(0:N))');
    bz = real(D * (b(:) .* T.^(0:N))');

% Plotting Frequency Response
fvtool(bd, ad);
title('Chebyshev Type I Low-Pass Filter');
```

Output-



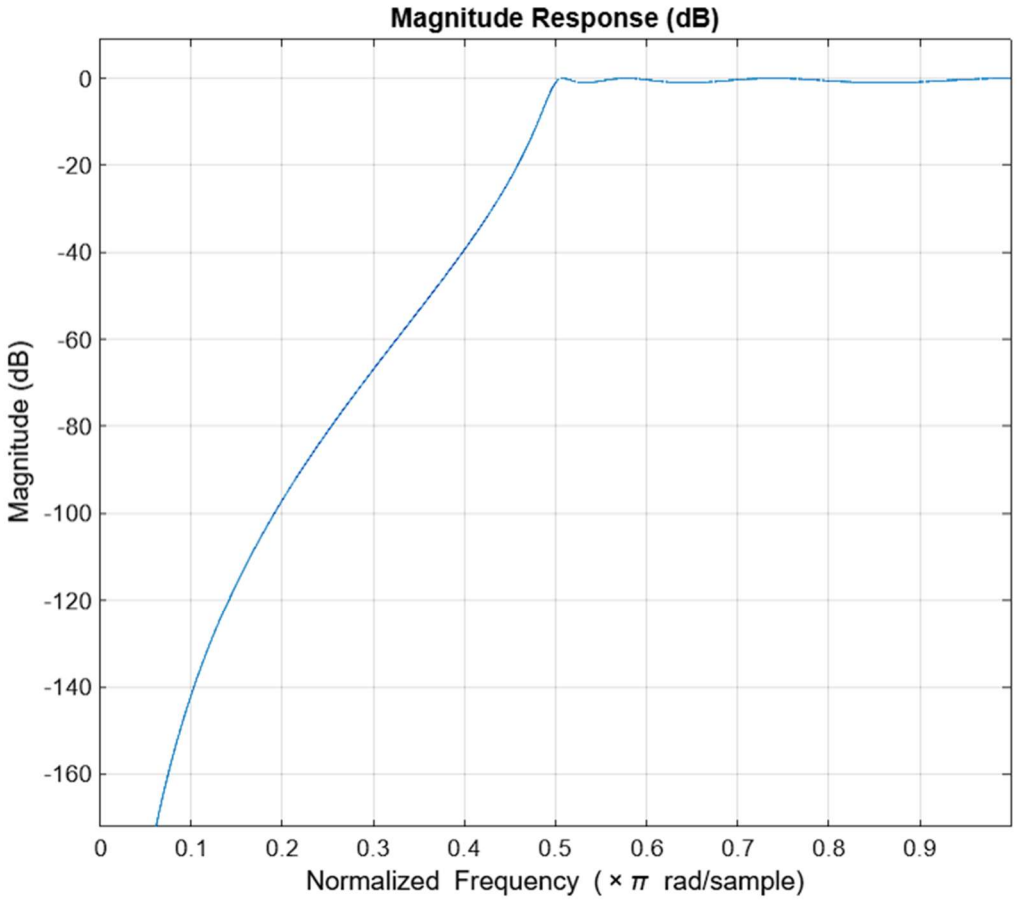
High Pass filter-

```
fs = 8000;          % Sampling frequency
fp = 2000;          % Passband frequency (Hz)
fsb = 1500;         % Stopband frequency (Hz)
Ap = 1;             % Passband attenuation (dB)
As = 40;            % Stopband attenuation (dB)

Wp = 2 * fs * tan(pi * fp / fs);  % Prewarped passband frequency
Ws = 2 * fs * tan(pi * fsb / fs); % Prewarped stopband frequency

ws = Wp / Ws;  % Normalized stopband frequency

% Compute Filter Order
ep = sqrt(10^(Ap/10) - 1);  % Ripple factor
A = 10^(As/20);             % Attenuation factor
n = acosh(sqrt((A^2 - 1)/ep^2)) / acosh(ws);  % Order formula
n = ceil(n);                % rounding up to nearest integer
beta = asinh(1/ep)/n;
p_lp = zeros(1, n);         % initializing pole

for k = 1:n
   theta = pi * (2*k - 1) / (2*n);      % Pole angle for each k
```

```
    sigma = -sinh(beta) * sin(theta);     % Real part of pole
    omega = cosh(beta) * cos(theta);      % Imaginary part of
pole
    p_lp(k) = sigma + 1j * omega;         % Scale poles by Wp
end

% Transform to High-Pass
p_hp = Wp ./ p_lp;
a_hp = real(poly(p_hp));
b_hp = real(a_hp(1) * ones(1, length(a_hp)));  % Constant
gain

% Bilinear transform
T = 1/fs;
n_ord = max(length(a_hp), length(b_hp)) - 1;

a_hp = [a_hp zeros(1, n_ord + 1 - length(a_hp))];
b_hp = [b_hp zeros(1, n_ord + 1 - length(b_hp))];

ahp = zeros(1, n_ord+1);
bhp = zeros(1, n_ord+1);

for k = 0:n_ord
    for i = k:n_ord
```

```
ahp(k+1) = ahp(k+1) + a_hp(i+1) * nchoosek(i, k) * (2/T)^(i - k);
bhp(k+1) = bhp(k+1) + b_hp(i+1) * nchoosek(i, k) * (2/T)^(i - k);
    end
end

% Normalizing the filter coefficients
bhp = bhp / ahp(1);
ahp = ahp / ahp(1);

% Plot Frequency Response
fvtool(bhp, ahp);
title('Chebyshev Type I High-Pass Filter');
```

Output-



2.

Low Pass Filter-

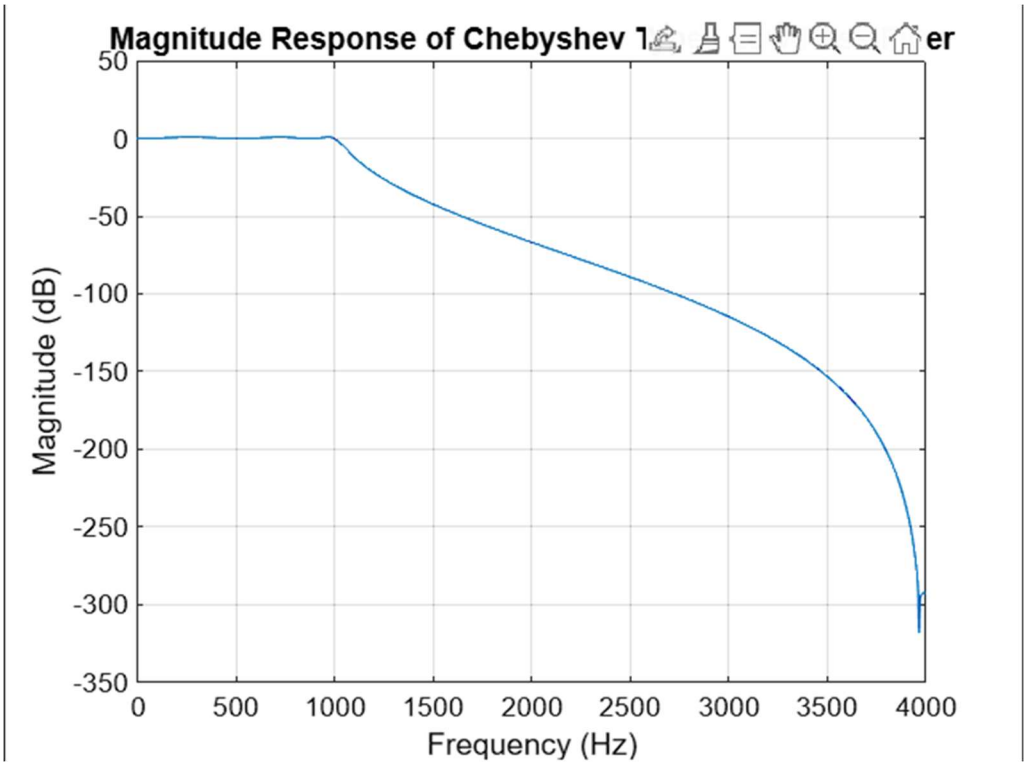
```
fs = 8000;          % Sampling frequency
fp = 1000;          % Passband frequency (Hz)
fsb = 1500;         % Stopband frequency (Hz)
Ap = 1;             % Passband attenuation (dB)
As = 40;            % Stopband attenuation (dB)
```

```
Wp = 2 * fs * tan(pi * fp / fs); % Prewarped passband frequency
Ws = 2 * fs * tan(pi * fsb / fs); % Prewarped stopband frequency
ws = Ws / Wp; % Normalized stopband frequency

ep = sqrt(10^(Ap/10) - 1); % Ripple factor
A = 10^(As/20);           % Attenuation factor
n = acosh(sqrt((A^2 - 1)/ep^2)) / acosh(ws);  % Order
n = ceil(n);  % Rounding up to nearest integer

beta = asinh(1/ep)/n;
p = zeros(1, n);    % initialize pole array
for k = 1:n
   theta = pi * (2*k - 1) / (2*n);   % pole angle
   sigma = -sinh(beta) * sin(theta);  % real part of pole
   omega = cosh(beta) * cos(theta);  % imaginary part of pole
   p(k) = Wp * (sigma + 1j * omega);   % scale poles by Wp
end

a = real(poly(p)); % denominator
b = [a(end)];  % numerator
```

```
%Bilinear Transform

T = 1/fs;
N = max(length(b), length(a)) - 1;
[b, a] = eqtflength(b, a);  % Ensure equal length

bd = zeros(1, N+1);
ad = zeros(1, N+1);

for n = 0:N
   for k = n:N
      bd(n+1) = bd(n+1) + b(k+1) * nchoosek(k,n) * (2/T)^(k-n)
* (-1)^(k-n);
      ad(n+1) = ad(n+1) + a(k+1) * nchoosek(k,n) * (2/T)^(k-n)
* (-1)^(k-n);
   end
end

% Normalize
bd = real(bd / ad(1));
ad = real(ad / ad(1));
% Computing the Frequency Response
f = linspace(0, fs/2, 1024);  % Frequency range from 0 to
Nyquist frequency
```

```
omega = 2 * pi * f / fs;  % Angular frequency

H = zeros(1, length(f));  % Initialize frequency response

% Loop over each frequency to calculate the frequency response
for i = 1:length(f)
    % Compute transfer function
    num = sum(bd .* z(i).^(-(0:length(bd)-1)));
    den = sum(ad .* z(i).^(-(0:length(ad)-1)));

    H(i) = num / den;  % Frequency response at this ω
end

% Plot the Magnitude Response in dB
figure;
plot(f, 20*log10(abs(H)));  % Plot magnitude in dB
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
title('Magnitude Response of Chebyshev Type I Low-Pass Filter');
grid on;
```

Output-



High Pass filter-

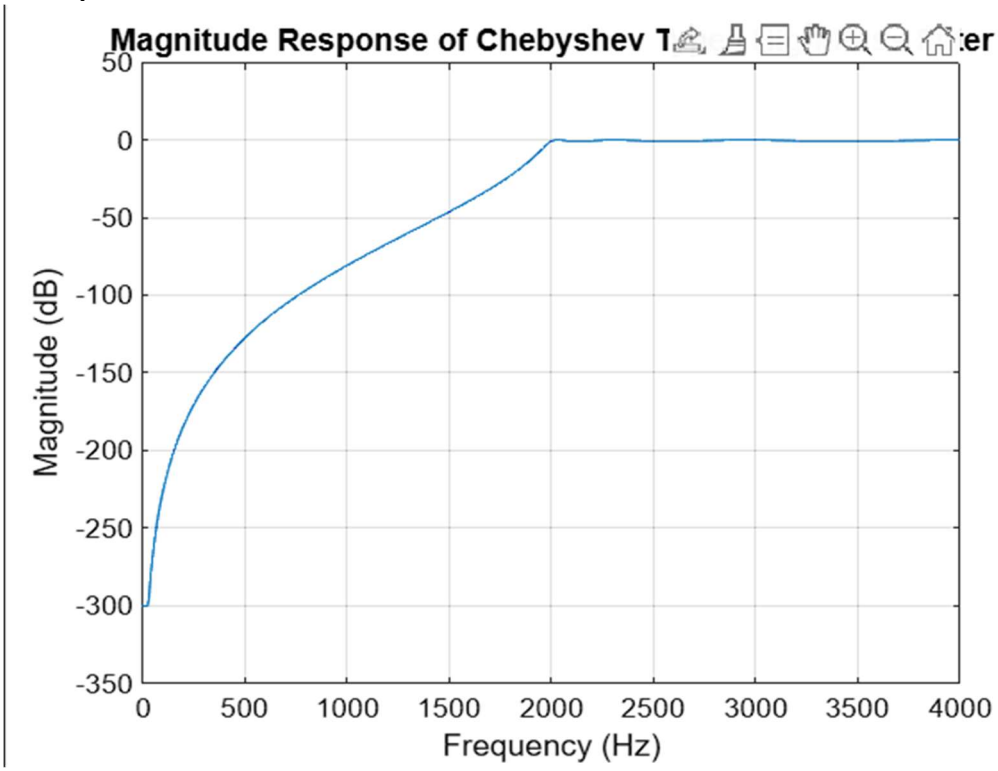
```
fs = 8000;          % Sampling frequency
fp = 2000;          % Passband frequency (Hz)
fsb = 1500;         % Stopband frequency (Hz)
Ap = 1;             % Passband attenuation (dB)
As = 40;            % Stopband attenuation (dB)

Wp = 2 * fs * tan(pi * fp / fs); % Prewarped passband frequency
```

```
Ws = 2 * fs * tan(pi * fsb / fs); % Prewarped stopband
frequency

ws = Wp / Ws; % Normalized stopband frequency

% Compute Filter Order
ep = sqrt(10^(Ap/10) - 1);  % Ripple factor
A = 10^(As/20);  % Attenuation factor
n = acosh(sqrt((A^2 - 1)/ep^2)) / acosh(ws);  % Order formula
n = ceil(n); % rounding up to nearest integer

beta = asinh(1/ep)/n;
p_lp = zeros(1, n);  % Initializing pole array
for k = 1:n
   theta = pi * (2*k - 1) / (2*n);  % Pole angle
   sigma = -sinh(beta) * sin(theta);  % Real part of pole
   omega = cosh(beta) * cos(theta);  % Imaginary part of pole
   p_lp(k) = sigma + 1j * omega;  % Scale poles by Wp
end

p_hp = Wp ./ p_lp;  % scaling poles for high-pass
a_hp = real(poly(p_hp));  % denominator coefficients
b_hp = real(a_hp(1) * ones(1, length(a_hp)));  % constant gain
```

```
%Bilinear Transform
T = 1/fs;
N = length(a_hp);
M = length(b_hp);
D = max(N, M) - 1;

a = [a_hp zeros(1, D - (N-1))];
b = [b_hp zeros(1, D - (M-1))];

bhp = zeros(1, D+1);
ahp = zeros(1, D+1);

for n = 0:D
   for k = n:D
      bhp(n+1) = bhp(n+1) + b(k+1)*nchoosek(k,n)*(2/T)^(k-n);
      ahp(n+1) = ahp(n+1) + a(k+1)*nchoosek(k,n)*(2/T)^(k-n);
   end
end

% Normalize
bhp = bhp / ahp(1);
ahp = ahp / ahp(1);
```

```
% Frequency Response
f = linspace(0, fs/2, 1024);  % Frequency range from 0 to Nyquist frequency
omega = 2 * pi * f / fs;      % Angular frequency
H = zeros(1, length(f));      % Initialize response

for i = 1:length(f)
    num = polyval(bhp, exp(1j*omega(i)));  % Numerator
    den = polyval(ahp, exp(1j*omega(i)));  % Denominator
    H(i) = num / den;  % Frequency response at ω
end
% Calculate Magnitude Response in dB
magnitude_dB = 20 * log10(abs(H));  % Convert magnitude to dB

% Plot the Magnitude Response
figure;
plot(f, magnitude_dB);  % Plot magnitude in dB
xlabel('Frequency (Hz)'); % X -axis
ylabel('Magnitude (dB)');%     Y-axis
title('Magnitude Response of Chebyshev Type I High-Pass Filter'); %title
grid on;
```

Output-



3.

Low Pass Filter-

```
fs = 8000;          % Sampling frequency (Hz)
fp = 1000;          % Passband frequency (Hz)
fsb = 1500;         % Stopband frequency (Hz)
Ap = 1;             % Passband attenuation (dB)
As = 40;            % Stopband attenuation (dB)

Wp = 2 * fs * tan(pi * fp / fs); % Prewarped passband
frequency
```

```
Ws = 2 * fs * tan(pi * fsb / fs); % Prewarped stopband frequency
ws = Ws / Wp; % Normalized stopband frequency

% Computing Filter Order (n)
ep = sqrt(10^(Ap/10) - 1); % Ripple factor
A = 10^(As/20);            % Attenuation factor

n = acosh(sqrt((A^2 - 1) / ep^2)) / acosh(ws);  % Order of the filter
n = ceil(n);  % Round up to nearest integer

beta = asinh(1 / ep) / n;
p = zeros(1, n);    % Initialize pole array

for k = 1:n
        theta = pi * (2 * k - 1) / (2 * n);  % pole angle
        sigma = -sinh(beta) * sin(theta);  % Real part of pole
        omega = cosh(beta) * cos(theta);  % Imaginary part of pole
       p(k) = Wp * (sigma + 1j * omega); %scale poles by Wb
end

%Analog low- pass transfer function
```

```
a = real(poly(p)); % Denominator
b = [a];  % Numerator

% Apply the bilinear transformation to map analog poles to
digital
T = 2 / fs;  % Bilinear transform constant
b_digital = b;  % digital numerator
a_digital = 1;  % digital denominator

for k = 1:n
    % Apply the bilinear transformation formula to each pole
    z = (1 + T * p(k)) / (1 - T * p(k));  % Digital pole

a_digital = conv(a_digital, [1 -2*real(z) abs(z)^2]);  %
Convolution for new denominator coefficients
end

% Displaying the filter coefficients (numerator and
denominator)
disp('Low-pass filter coefficients:');
disp('Numerator (b): ');
disp(b_digital);
disp('Denominator (a): ');
disp(a_digital);
```

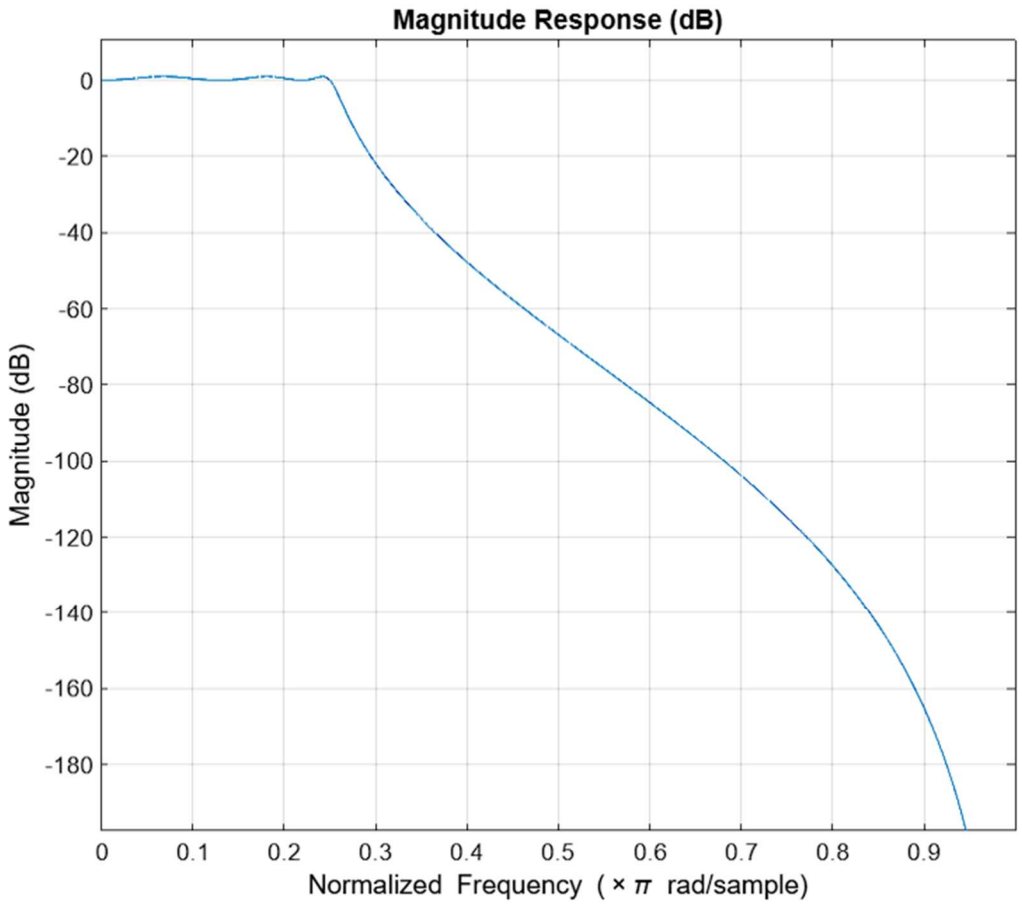
Output-

Numerator (b):

0.0000 0.0000 0.0000 0.0000 0.0000 0.0039 5.8389

Denominator (a):

1.0000 1.1928 3.0433 1.7118 2.9271 0.5366 1.5887



High Pass Filter-

```
Fs = 8000;          % Sampling frequency (Hz)
fp = 2000;          % Passband frequency (Hz)
fsb = 1500;         % Stopband frequency (Hz)
Ap = 1;             % Passband ripple (dB)
As = 40;            % Stopband attenuation (dB)

Wp = 2 * Fs * tan(pi * fp / Fs);   % Prewarped passband frequency
Ws = 2 * Fs * tan(pi * fsb / Fs);  % Prewarped stopband frequency

ws = Wp / Ws;  % Normalized stopband frequency

ep = sqrt(10^(Ap/10) - 1);     % ripple factor
A = 10^(As/20);                % Attenuation factor
n = acosh(sqrt((A^2 - 1) / ep^2)) / acosh(ws);  % Filter order
n = ceil(n);  % Round up to the next integer

beta = asinh(1 / ep) / n;
p_lp = zeros(1, n);
```

```
for k = 1:n
    theta = pi * (2*k - 1) / (2*n);
    sigma = -sinh(beta) * sin(theta);
    omega = cosh(beta) * cos(theta);
    p_lp(k) = sigma + 1j * omega;
end

p_hp = Wp ./ p_lp;  % High-pass poles

a_hp = real(poly(p_hp));          % Denominator
b_hp = real(a_hp(1) * ones(1, n+1));  % Numerator

%Bilinear Transform
T = 1 / Fs;
M = length(b_hp);
N = length(a_hp);
order = max(M, N);

% Pad coefficients to equal length
b_pad = [b_hp zeros(1, order - M)];
a_pad = [a_hp zeros(1, order - N)];

% Initialize transformed coefficients
```

```
bhp = zeros(1, order);
ahp = zeros(1, order);

% Bilinear transform (impulse invariance)
for i = 0:(order-1)
   for k = i:(order-1)
      bhp(i+1) = bhp(i+1) + b_pad(k+1) * nchoosek(k, i) *
(2/T)^(k-i);
      ahp(i+1) = ahp(i+1) + a_pad(k+1) * nchoosek(k, i) *
(2/T)^(k-i);
   end
end

% Normalize
bhp = bhp / ahp(1);
ahp = ahp / ahp(1);

disp('Numerator coefficients (bhp):');
disp(bhp);
disp('Denominator coefficients (ahp):');
disp(ahp);

% Plot frequency response using FVTool
fvtool(bhp, ahp);
```

```
title('Chebyshev Type I High-Pass Filter');
```

Output-

Numerator coefficients (bhp):

0.0040 -0.0280 0.0840 -0.1400 0.1400 -0.0840 0.0280 -0.0040

Denominator coefficients (ahp):

1.0000 1.9907 3.4175 3.7926 3.2723 2.0592 0.9010 0.2365